TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Topic – DBMS

Lecture No.- 15

By- Rahul Sir

Topics to be Covered

Topic

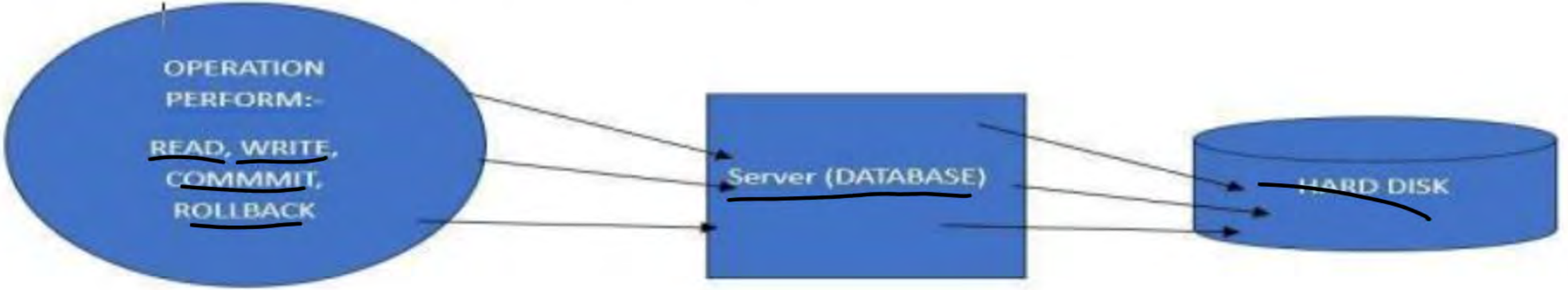
1:- TRANSCATION and ACID PROPERTY
2:- TRANSCATION STATE
3:- SCHDULE AND ITS TYPE
4:- CONCURRENCY PROBLEMS
5:- DIFFERENT PROTOCOL
6:- RECOVERABLE AND IRRECOVERABLE SCHDULE
7:- SEARIALIZIBILTY
8:-TYPES OF SEARIALIZIBILTY
9:-REVISION ON SCHDULE

Topic : TRANSCATION

TEACHING WALLAH

What is Transaction in DBMS?

It is an action or sequence of actions passed out by a single user and/or application program that reads or updates the contents of the database. A transaction is a logical piece of work of any database, which may be a complete program, a fraction of a program, or a single command (like the: SQL command INSERT or UPDATE) that may involve any number of processes on the database. From the database point of view, the implementation of an application program can be considered as one or more transactions with non-database processing working in between.



Topic : TRANSCATION

TEACHING WALLAH

R(A) ->1000

A=A-500

W(A)->500

R(B)->2000

B=B+500

W(B) -> 2500

COMMIT

failed

RAM A -500 B +500

RAM

CPU RAM temporary memory HD

Previous Data->

RAM

A=1000

B=2000

Final Data->

fail HD

A=500

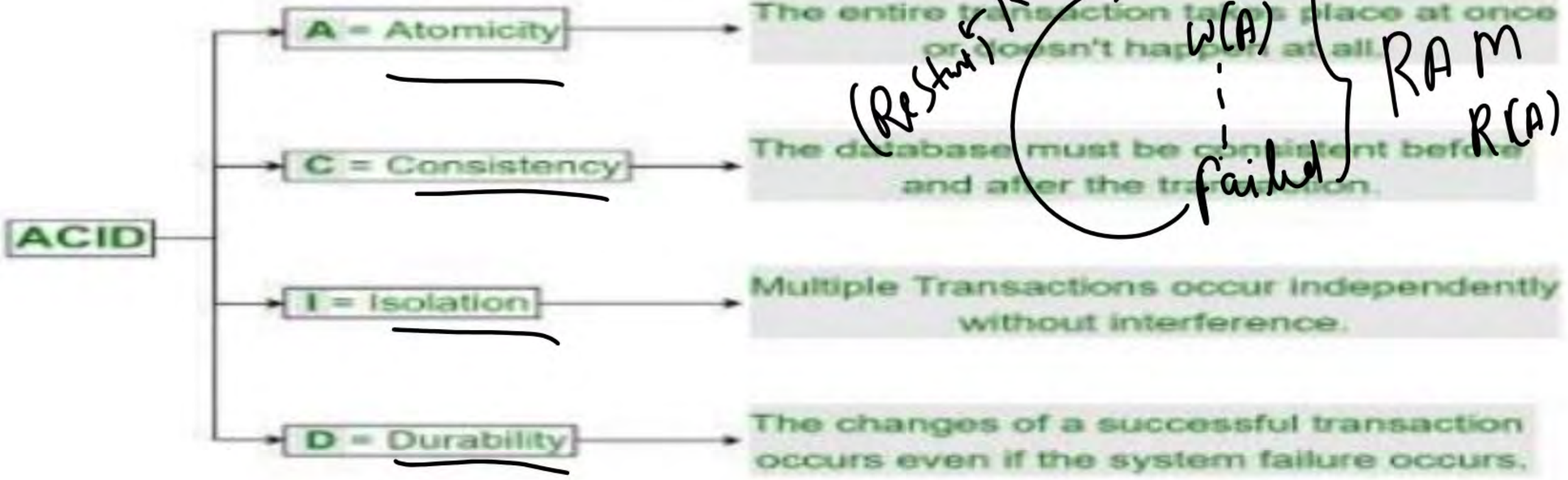
B=2500

Topic : ACID PROPERTY IN TRANSACTION

TEACHING WALLAH

ACID stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability**. These four key properties define how a transaction should be processed in a reliable and predictable manner, ensuring that the database remains consistent, even in cases of failures or concurrent accesses.

ACID Properties in DBMS



Topic : ATOMICITY

1. Atomicity: "All or Nothing"

Atomicity ensures that a transaction is **atomic**, it means that either the entire transaction completes fully or doesn't execute at all. There is no in-between state i.e. transactions do not occur partially. If a transaction has multiple operations, and one of them fails, the whole transaction is rolled back, leaving the database unchanged. This avoids partial updates that can lead to inconsistency.

- **Commit**: If the transaction is successful, the changes are permanently applied.
- **Abort/Rollback**: If the transaction fails, any changes made during the transaction are discarded.

Topic : ATOMICITY

TEACHING WALLAH

Example: Consider the following transaction T consisting of **T1** and **T2** : Transfer of $100 from account **X** to account **Y** .

Before: X : 500	Y : 200
Transaction T	
T1	T2
Read (X) X: = X - 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

Commit Commit

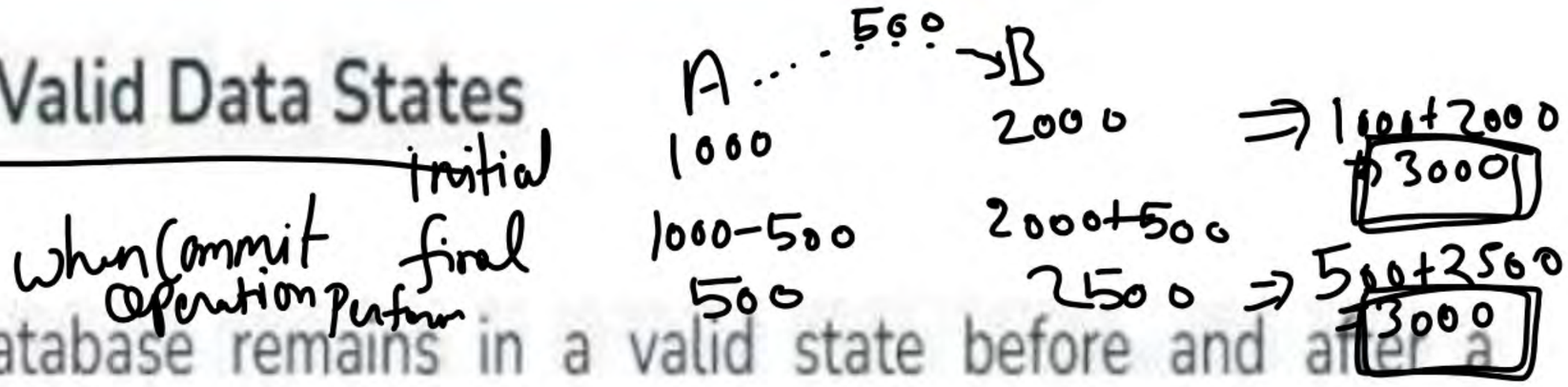
Example

If the transaction fails after completion of T1 but before completion of T2 , the database would be left in an inconsistent state. With Atomicity, if any part of the transaction fails, the entire process is rolled back to its original state, and no partial changes are made.

Topic : CONSISTENCY

TEACHING WALLAH

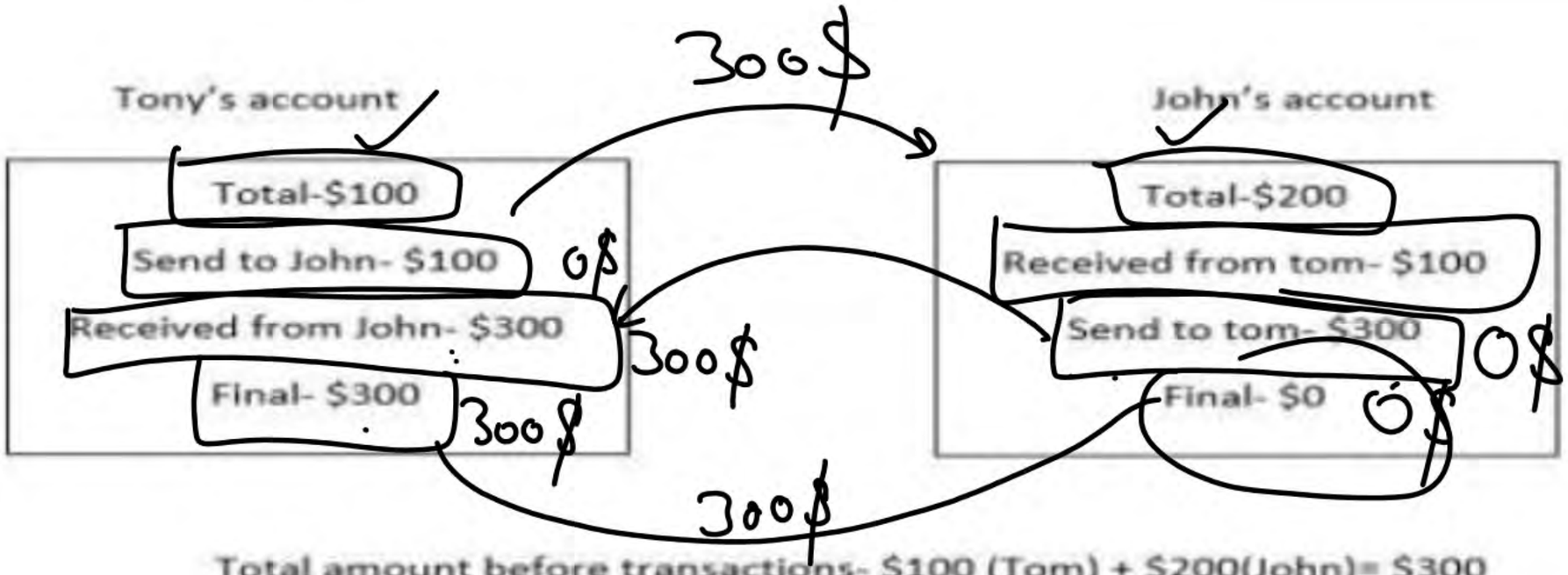
2. Consistency: Maintaining Valid Data States



Consistency ensures that a database remains in a valid state before and after a transaction. It guarantees that any transaction will take the database from one consistent state to another, maintaining the rules and constraints defined for the data. In simple terms, a transaction should only take the database from one **valid** state to another. If a transaction violates any database rules or constraints, it should be rejected, ensuring that only consistent data exists after the transaction.

Topic : CONSISTENCY

Let's take an example. Tony and John have $100 and $200 in their bank account: respectively. The total of their amount is $300. So, even after the thousands of transactions between them, the total amount should remain $300. If any problem occurs, like system failure, the total amount must not change. Otherwise, we can say the database is not consistent.



Total amount before transactions- $100 (Tom) + $200(John)= $300

Total amount after transactions- $300 (Tom) + $0 (John) = $300

Topic : ISOLATION

Isolation property ensures that no two transactions of the same database interfere with each other. If two transactions are running concurrently on the same database, the second transaction must not read the values from the database until the first transaction is completed.

In short, the second transaction should proceed only when the first transaction is completed, and the changes are committed in the memory.

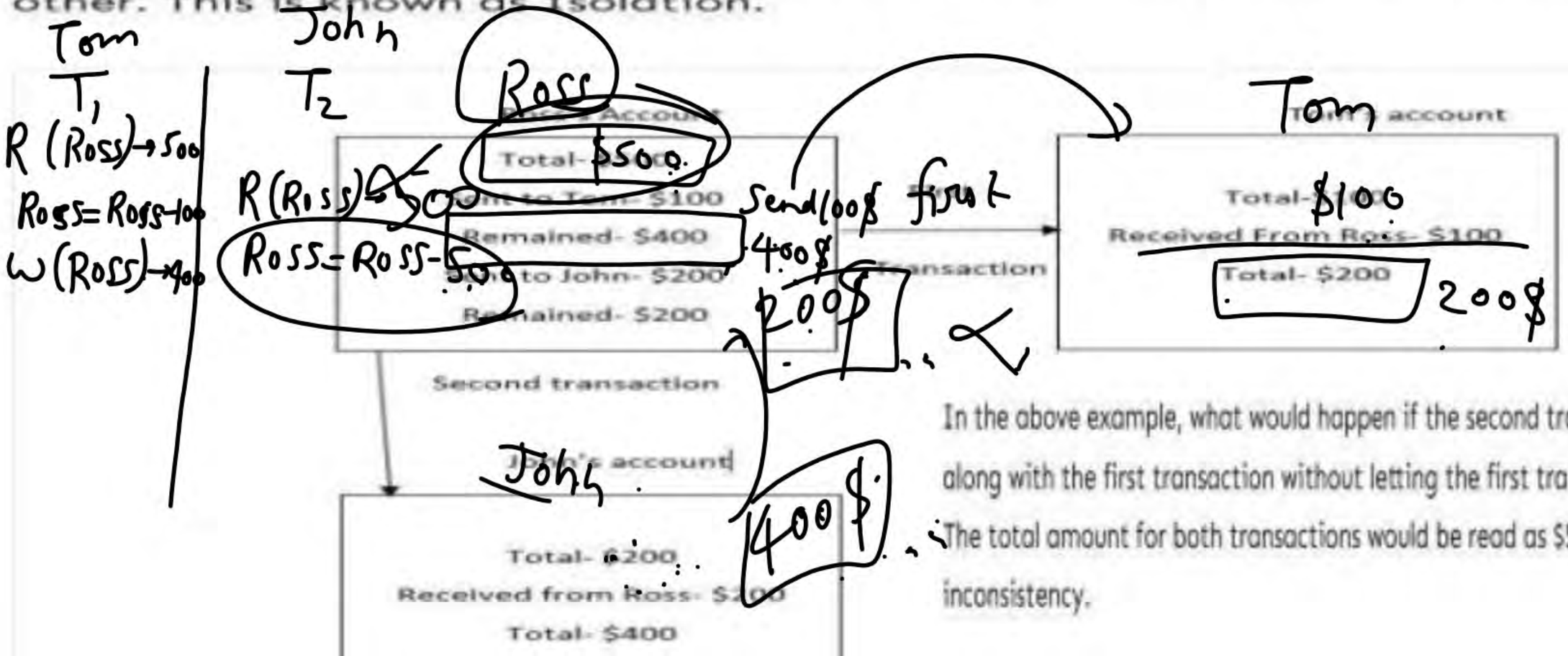
Isolation property can be explained with a simple example. Let's say we have three bank accounts of Tom, John, and Ross with the amount $100, $200, and $500.

Now, Ross needs to send $100 to Tom and $200 to John. Here, we have two independent transactions that must not interfere with each other.

Topic : ISOLATION

When the first transaction is completed (After sending $100 to Tom), Ross will have $400 in his account, and then he will send $200 to John and end up with $200.

Here, both transactions are executed independently without affecting each other. This is known as Isolation.



In the above example, what would happen if the second transaction takes place along with the first transaction without letting the first transaction complete?

The total amount for both transactions would be read as $500, resulting in inconsistency.

Topic : DURABILITY

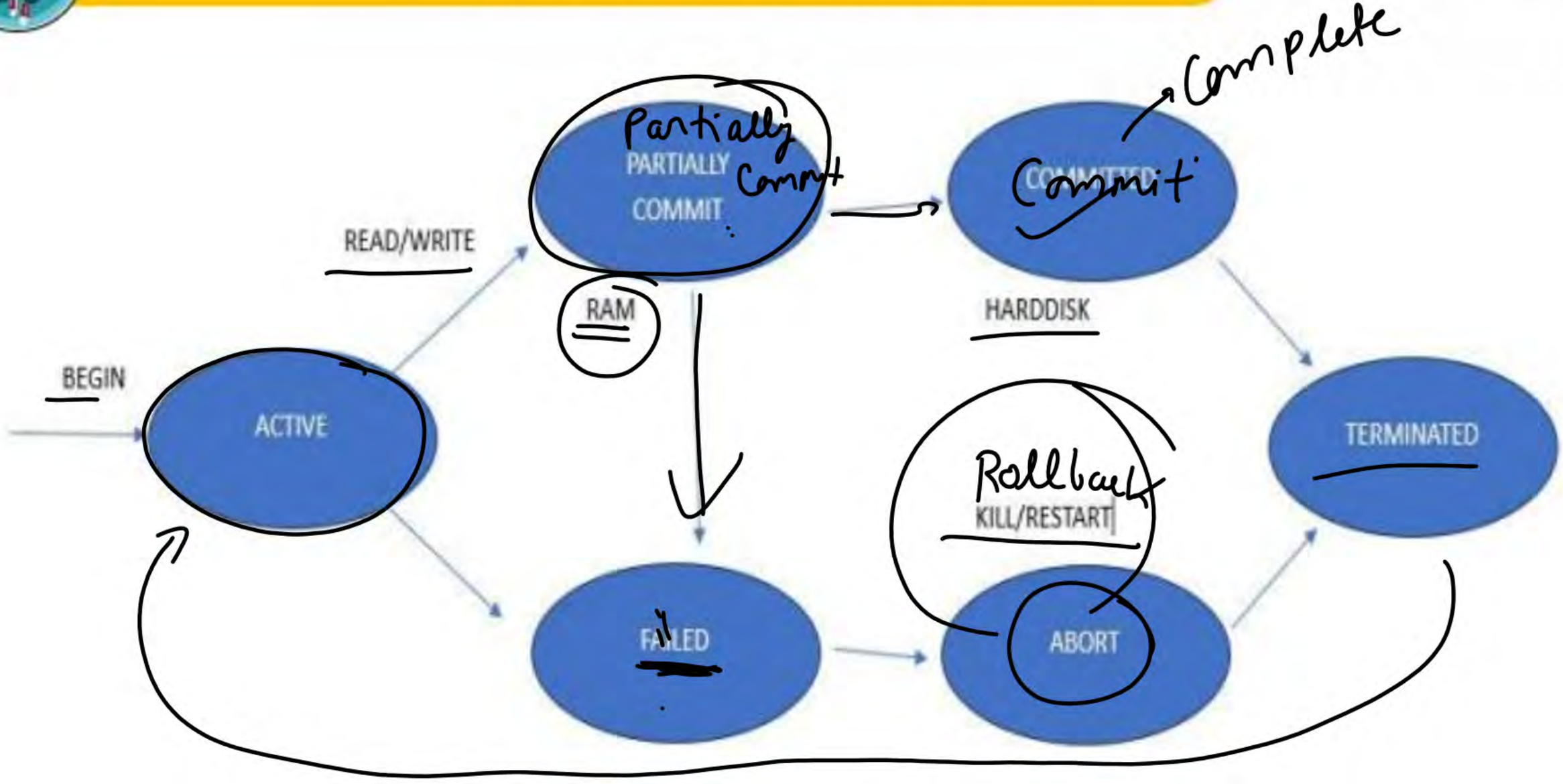
4. Durability: Persisting Changes

This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs. These updates now become permanent and are stored in **non-volatile memory**. In the event of a failure, the DBMS can recover the database to the state it was in after the last committed transaction, ensuring that no data is lost.

Example: After successfully transferring money from Account A to Account B, the changes are stored on disk. Even if there is a crash immediately after the commit, the transfer details will still be intact when the system recovers, ensuring durability.

Topic : TRANSACTION STATE

TEACHING WALLAH



Topic : SCHDULE

Transactions are a set of instructions that perform operations on databases. When multiple transactions are running concurrently, then a sequence is needed in which the operations are to be performed because *at a time, only one operation can be performed on the database*. **This sequence of operations is known as Schedule, and this process is known as Scheduling**

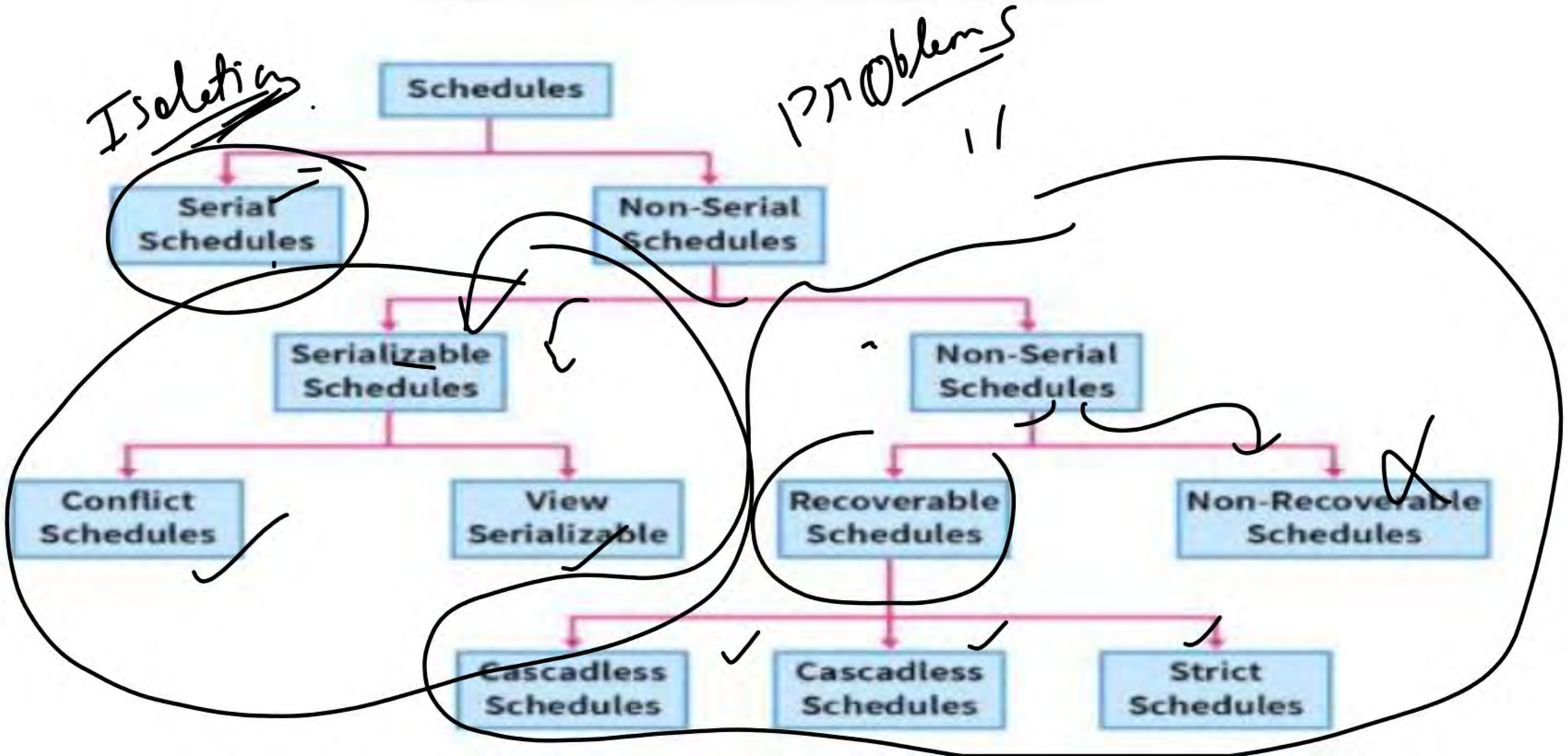
why is it required?

When multiple transactions execute simultaneously in an unmanageable manner, then it might lead to several problems, which are known as concurrency problems. In order to overcome these problems, scheduling is required.

Topic : SCHDULE

TEACHING WALLAH

Types of Schedules in DBMS



Topic : All Concurrency Problems

TEACHING WALLAH

Topic : Dirty Read /uncommitted read or RAW

TEACHING WALLAH

This read is called as dirty read because-

- There is always a chance that the uncommitted transaction might roll back later.
- Thus, uncommitted transaction might make other transactions read a value that does not even exist.
- This leads to inconsistency of the database.

NOTE-

- Dirty read does not lead to inconsistency always.
- **It becomes problematic only when the uncommitted transaction fails and roll backs later due to some reason**

Topic : Dirty Read /uncommitted read or RAW

Topic : UNREPEATTED READ

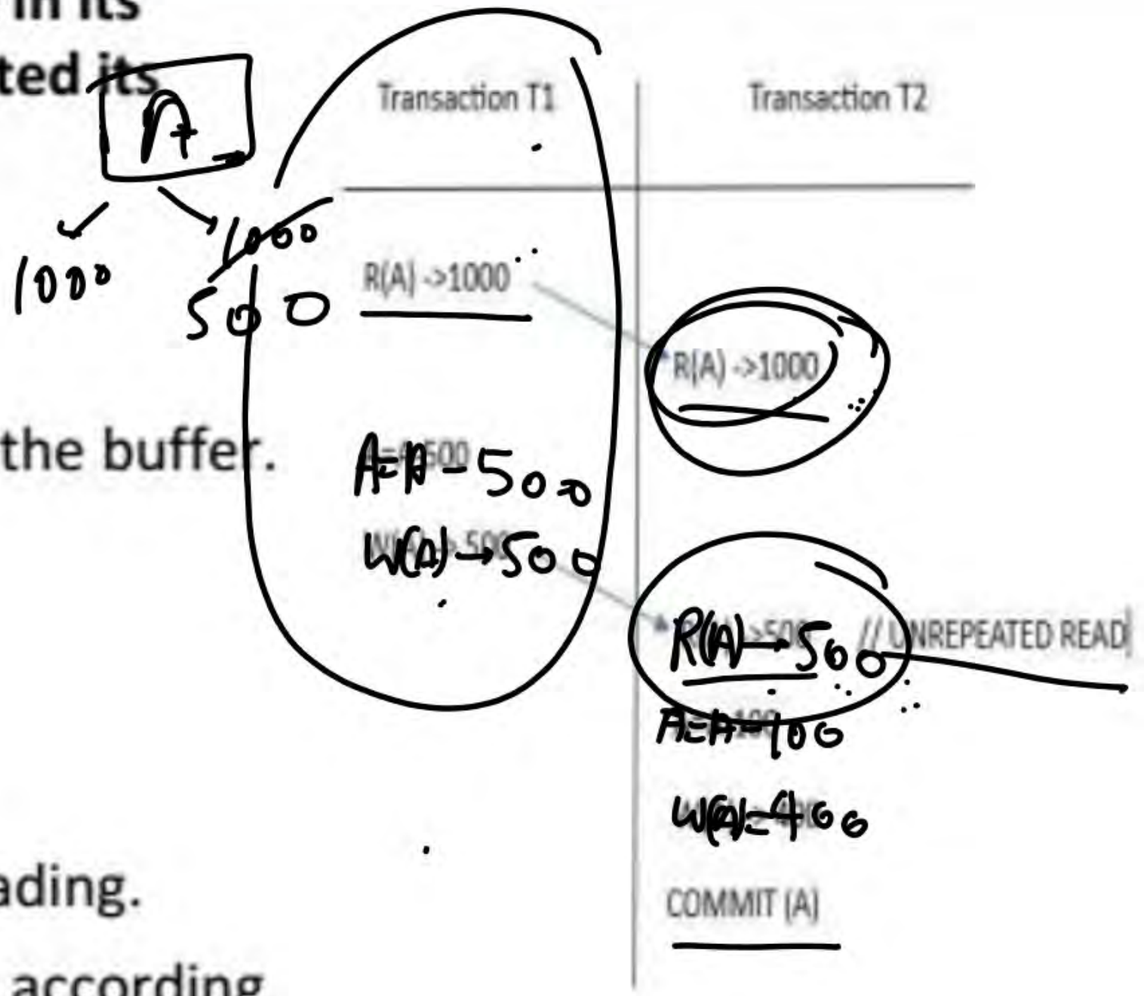
This problem occurs when a transaction gets to read unrepeated i.e. different values of the same variable in its different read operations even when it has not updated its value

1.T1 reads the value of X (= 1000 say).

2.T2 reads the value of X (= 1000).

3.T1 updates the value of X (from 1000 to 500 say) in the buffer.

4.T2 again reads the value of X (but = 500).



In this example,

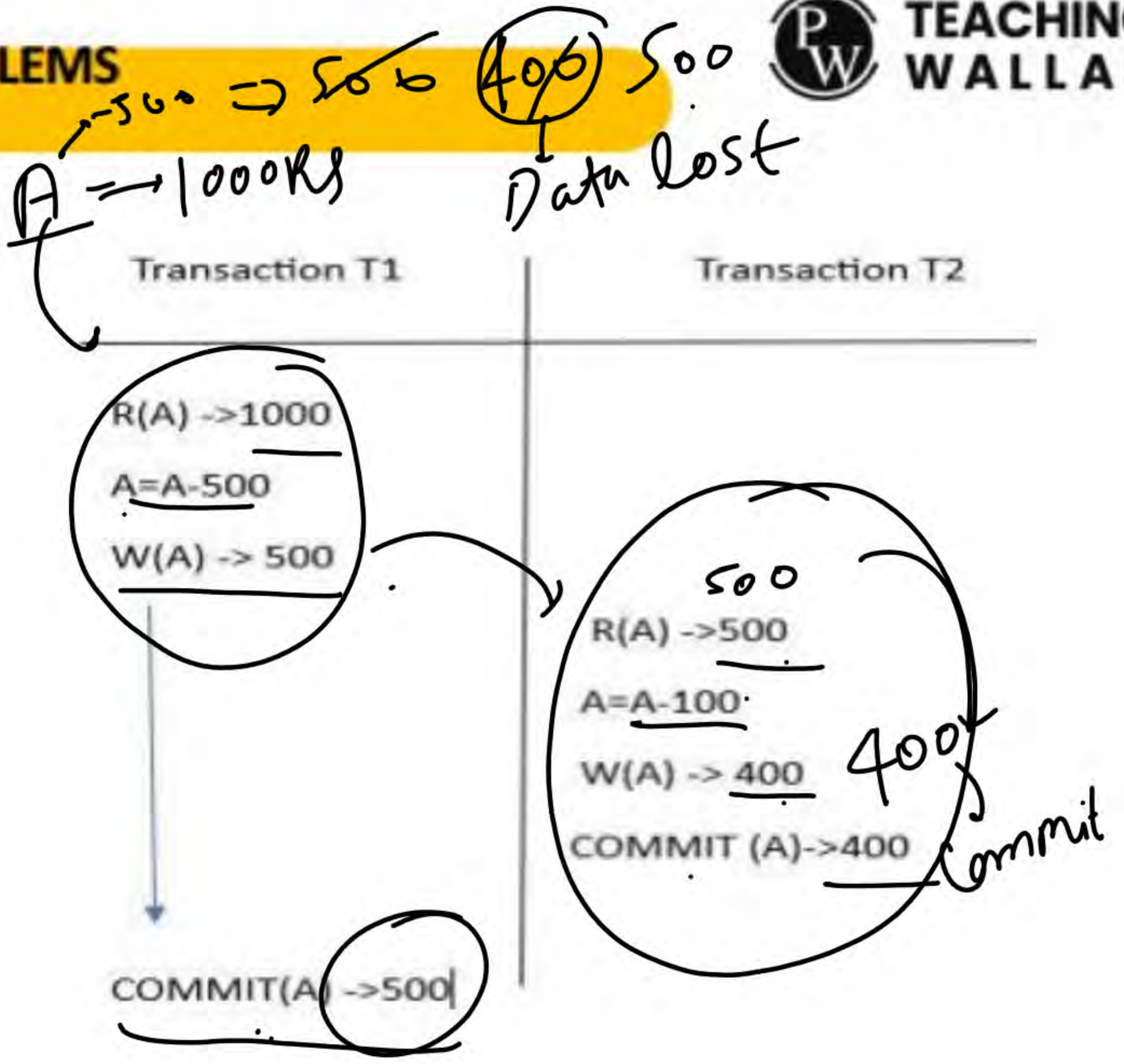
•T2 gets to read a different value of X in its second reading.

•T2 wonders how the value of X got changed because according to it, it is running in isolation.

Topic : LOST UPDATE PROBLEMS

TEACHING WALLAH

This problem occurs when multiple transactions execute concurrently and updates from one or more transactions get lost.



Topic : LOST UPDATE PROBLEMS

TEACHING WALLAH

1.T1 reads the value of A (= 1000 say) and updated write = 500

2.T2 read the value 500 and updates the value to A (= 400 say) in the buffer.

3.T2 commits.

4.When T1 commits, it already writes A = 400 in the database because of T2 COMMIT EARILER.

In this example,

- T1 writes the over written value of A in the database.
- Thus, update from T2 gets lost.

Topic : Phantom Read Problem

TEACHING WALLAH

This problem occurs when a transaction reads some variable from the buffer and when it reads the same variable later, it finds that the variable does not exist

1.T1 reads X.

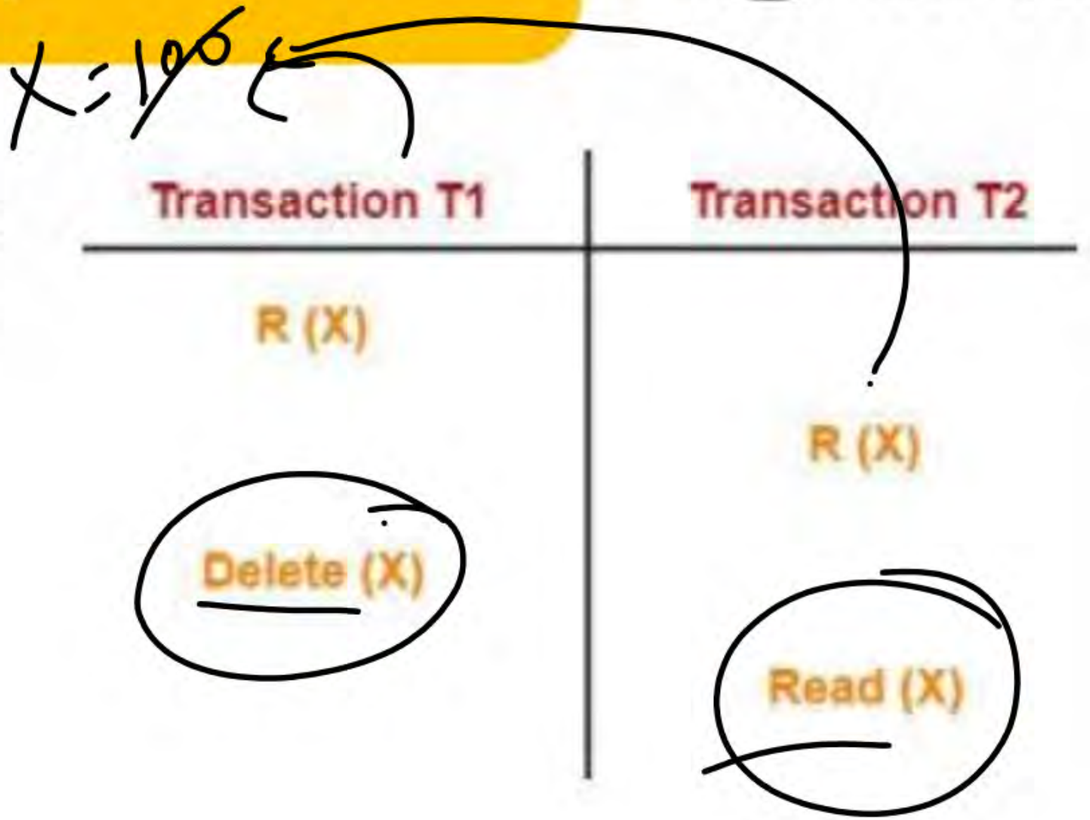
2.T2 reads X.

3.T1 deletes X.

4.T2 tries reading X but does not find it.

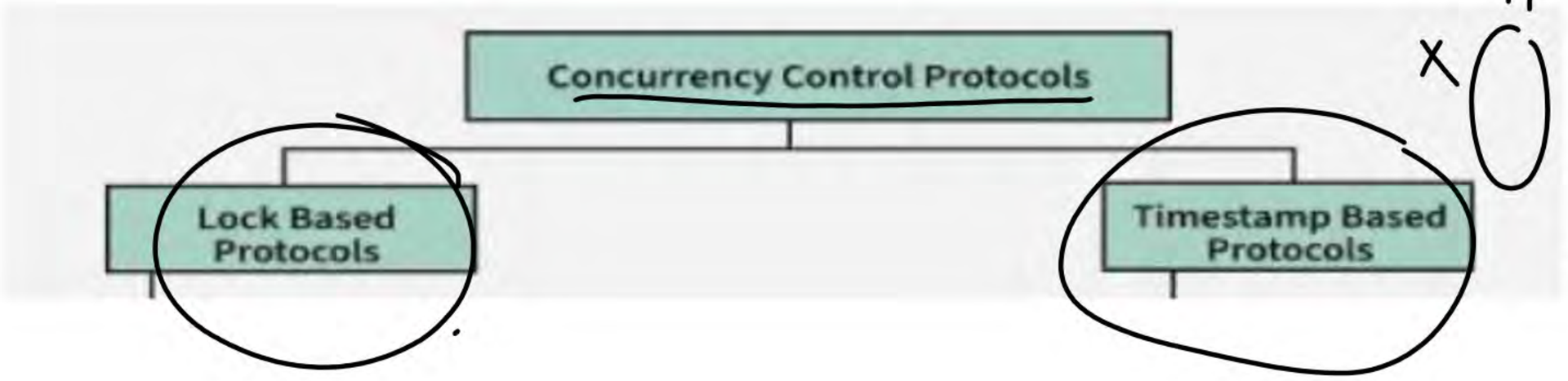
In this example,

- T2 finds that there does not exist any variable X when it tries reading X again.
- T2 wonders who deleted the variable X because according to it, it is running in isolation



Topic : Conclusion of Concurrency Problem in Parallel Schedule for same Variable

- To ensure consistency of the database, it is very important to prevent the occurrence of above problems.
- **Concurrency Control Protocols** help to prevent the occurrence of above problems and maintain the consistency of the database.



Topic :LOCK BASED PROTOCOL

TEACHING WALLAH

What is a Lock?

A lock is a variable associated with a data item that indicates whether it is currently in use or available for other operations. Locks are essential for managing access to data during concurrent transactions. **When one transaction is accessing or modifying a data item, a lock ensures that other transactions cannot interfere with it, maintaining data integrity and preventing conflicts. This process, known as locking**, is a widely used method to ensure smooth and consistent operation in database systems.

Lock Based Protocols

Lock-Based Protocols in DBMS ensure that a transaction cannot read or write data until it gets the necessary lock. Here's how they work:

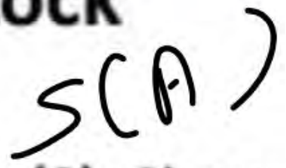
- These protocols prevent concurrency issues by allowing only one transaction to access a specific data item at a time.

- Locks help multiple transactions work together smoothly by managing access to the database items.

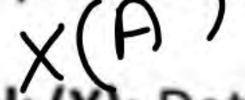
- Locking is a common method used to maintain the serializability of transactions.

Topic :LOCK BASED PROTOCOL

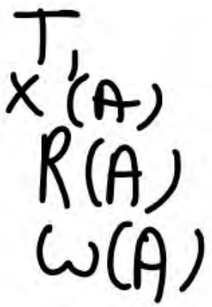
Types of Lock



1.Shared Lock (S): Shared Lock is also known as Read-only lock. As the name suggests it can be shared between transactions because while holding this lock the transaction does not have the permission to update data on the data item. S-lock is requested using lock-S instruction.



2.Exclusive Lock (X): Data item can be both read as well as written. This is Exclusive and cannot be held simultaneously on the same data item. X-lock is requested using lock-X instruction.



Topic :LOCK BASED PROTOCOL

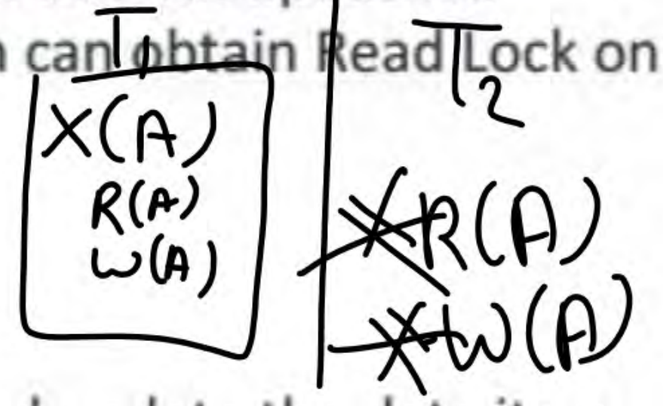
TEACHING WALLAH

Rules of Locking

The basic rules for Locking are given below :

Read Lock (or) Shared Lock(S)

- If a Transaction has a Read lock on a data item, it can read the item but not update it.
- If a transaction has a Read lock on the data item, other transaction can obtain Read Lock on the data item but no Write Locks.
- So, the Read Lock is also called a Shared Lock.



X(A) X(Z) X(User)

Write Lock (or) Exclusive Lock (X)

- If a transaction has a write Lock on a data item, it can both read and update the data item.
- If a transaction has a write Lock on the data item, then other transactions cannot obtain either a Read lock or write lock on the data item.
- So, the Write Lock is also known as Exclusive Lock.

Topic :LOCK BASED PROTOCOL

TEACHING WALLAH

Lock Compatibility Matrix

- A transaction can acquire a lock on a data item only if the requested lock is compatible with existing locks held by other transactions.
- **Shared Locks (S):** Multiple transactions can hold shared locks on the same data item simultaneously.
- **Exclusive Lock (X):** If a transaction holds an exclusive lock on a data item, no other transaction can hold any type of lock on that item.
- If a requested lock is not compatible, the requesting transaction must wait until all incompatible locks are released by other transactions.
- Once the incompatible locks are released, the requested lock is granted.

T_1 S(A) | T_2 S(A) | S(A) | X(A) R(A) W(A)

	S	X
S	✓	✗
X	✗	✗

Topic :LOCK BASED PROTOCOL

TEACHING WALLAH

Problems may occur on Serializability

Transaction T1	Transaction T2
X(A) – EXCLUSIVE ON A	
R(A)	
W(A)	
U(A)	
	S(A) – SHARED MODE ON A
	R(A)
X(B)- EXCLUSIVE ON B	
R(B)	
W(B)	

(Serializability problems)

WITHOUT UNLOCK OF EXCLUSIVE ON A T1 TRANSACTION HOW SHARED MODE ALLOCATE TO A -T2 TRANSACTION

WITHOUT UNLOCK OF A HOW IT GOES TO TRANSCATION T1

Topic :LOCK BASED PROTOCOL

TEACHING WALLAH

Solution :- problems may solve because of Unlock on variable
but Serializability means once T1 complete transaction then T2 will do Transaction

Transaction T1	Transaction T2
X(A)- EXCLUSIVE ON A	
R(A)	
W(A)	
U(A) – UNLOCK A	X(A)
	S(A) – SHARED MODE ON A
	R(A)
	W(A)
	U(A)– UNLOCK A
X(B)- EXCLUSIVE ON B	
R(B)	
W(B)	
U(B) – UNLOCK ON B	

Topic : 2 phase Locking (2 PL)

A transaction is said to follow the Two-Phase Locking protocol if Locking and Unlocking can be done in two phases :

- **Growing Phase:** New locks on data items may be acquired but none can be released.

- **Shrinking Phase:** Existing locks may be released but no new locks can be acquired.

Topic : 2 phase Locking (2 PL)

TEACHING WALLAH

Transaction T1	Transaction T2
X(A)	
R(A)	
W(A)	GROWING STAGE
X(B)	
R(B)	
W(B)	
U(A)	SHRINKING STAGE
U(B)	
	X(A)
	R(A)
	W(A)
	U(A) → SHRINKING STAGE

HERE YOU SEE (T1→T2) OR (T2->T1)

HERE WE SOLVE SERIALIZIBILITY PROBLEMS

GROWING STAGE

Topic : Timestamp Ordering Protocol

The **Timestamp Ordering Protocol** arranges the transactions based on their respective **timestamps**. A timestamp is a **numeric value assigned** to a transaction when it arrives in a schedule. Usually, this timestamp value is assigned in ascending order.

Example: Suppose three transactions, T1, T2, and T3, execute in parallel on the same data A.

- "T1 arrived at 8:00 AM. Please assign a timestamp value of 100 and call it '**Older**.'
- "T2 arrived at 8:10 AM, so please assign a timestamp value of 200 and name it '**Younger**.'
- "T3 arrived at 8:15 AM. Therefore, assign a timestamp value of 300 and name it '**Youngest**.'

Always execute the transaction that **comes first or has the minimum timestamp value.**

Topic : Types Of Timestamp Ordering Protocol

TEACHING WALLAH

The timestamp of any data is of two types

1. Read timestamp (RTS)
2. Write timestamp (WTS)

RTS = 300
WTS =

1. Read the Timestamp of the Data

The Read timestamp refers to the timestamp of the last successful read operation on data A.

> It is denoted by RTS ("data") = timestamp-Value.

Example:

T1 (Timestamp =100)	T2 (Timestamp =200)	T3 (Timestamp =300)
R(A)		
	R(A)	
		R(A)

According to the above diagram, data "A" was last read by transaction T3. Therefore, the read timestamp for data "A" will be the same as that of the least recent transaction.

> RTS ("A") = 300

Topic : Types Of Timestamp Ordering Protocol

TEACHING WALLAH

2. Write Timestamp of Data

The Write timestamp is the timestamp of the latest successful write operation on the same data.

WTS = 200

> It is denoted by WTS (Ti) = timestamp-Value.

Example:

T1 (Timestamp =100)	T2 (Timestamp =200)	T3 (Timestamp =300)
W(A)		
		W(A)
	W(A)	

As shown in the diagram, T2 is the most recent transaction that wrote data "A," thus establishing the write timestamp of the data, which is given below.

Topic : Types Of Timestamp Ordering Protocol

TEACHING WALLAH

If multiple data items are in the schedule (i.e., A, B, C...), each data item holds its own Read and Write timestamp as given below.

A
RTS = 300
WTS = 200

B
RTS = 100
WTS = 100

T1 (Timestamp = 100)	T2 (Timestamp = 200)	T3 (Timestamp = 300)
R(A)		R(B)
	R(A)	W(B)
R(B)	W(A)	R(A)
W(B)		

- RTS(A) = 300
- WTS(A) =200
- RTS(B) =100
- WTS(B) =100

Topic:- Rule for Timestamp Ordering Protocol

TEACHING WALLAH

Timestamps follow some rules to perform read or write operations. The rules are also known as Thomas rules.

Rule No. 01 is utilized when a transaction requires a Read (A) operation.

- If WTS(A) > TS (Ti), then Ti Rollback
- Else (otherwise) execute R(A) operation and **SET** RTS (A) = MAX {RTS(A), TS(Ti)}

Rules No.2 rules are used when a transaction needs to perform **WRITE (A)**

- If RTS(A) > TS (Ti), then Ti Rollback.
- If WTS(A) > TS (Ti), then Ti Rollback.
- Else (otherwise) execute the W(A) operation and **SET** WTS (A) = TS(Ti).

Where **"A"** is some data

NOTE:- We have to give preference to older Time stamp for performing R-W W-R W-W R-R Operation.

Topic:- Rule for Timestamp Ordering Protocol

TEACHING WALLAH

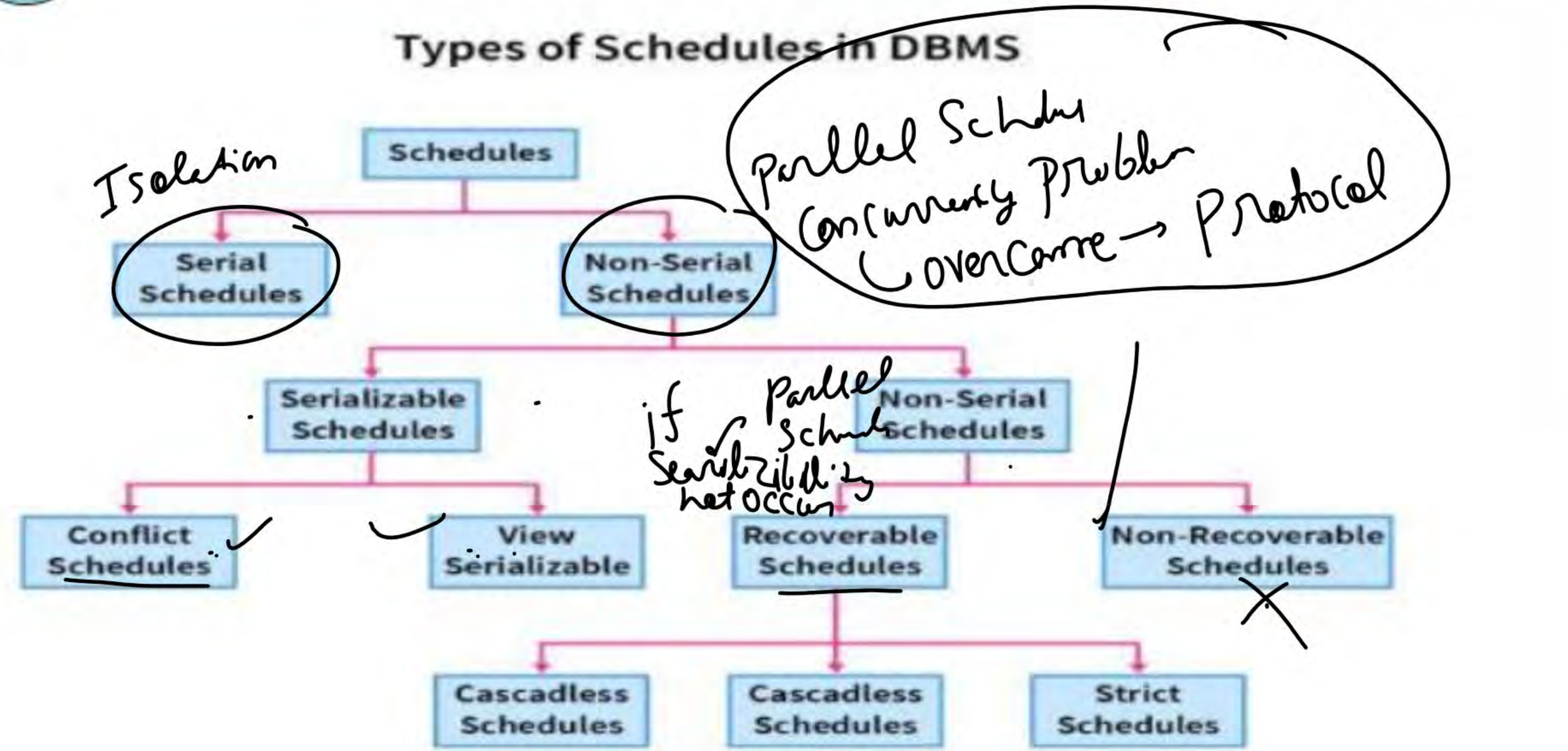
NOTE:- We have to give preference to older Time stamp for performing R-W,W-R,W-W,R-R Operation

	Transaction T1	Transaction T2	
	timestamp=100	timestamp=101	
Case-I	W(A)		
		R(A)	VALID
	R(B)		
Case-II		W(B)	VALID
	W(C)		
Case-III		W(C)	VALID
		W(D)	
Case-IV	W(D)		
			INVALID

Topic:- Short revision on schdule

TEACHING WALLAH

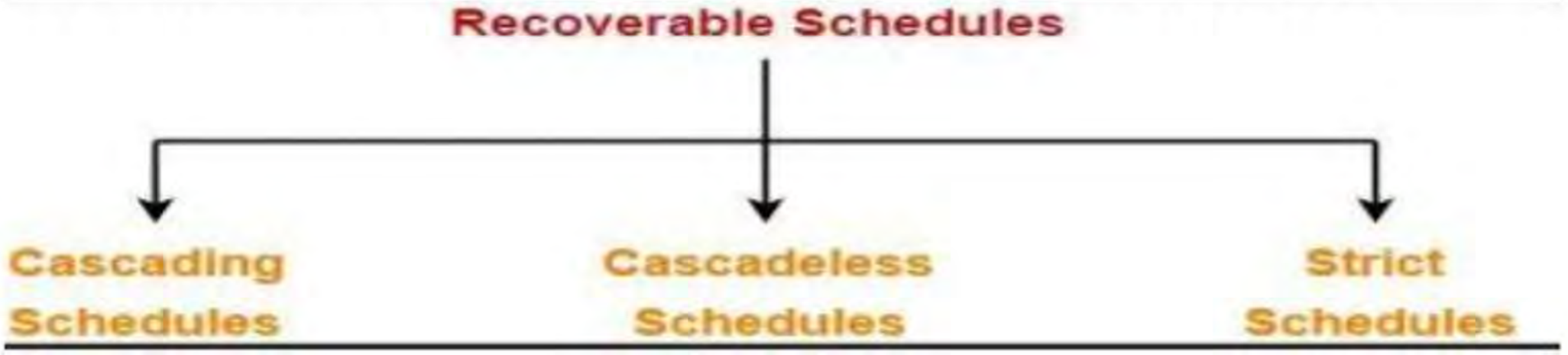
Types of Schedules in DBMS



Topic:- Recoverability and Recoverable Schedule

TEACHING WALLAH

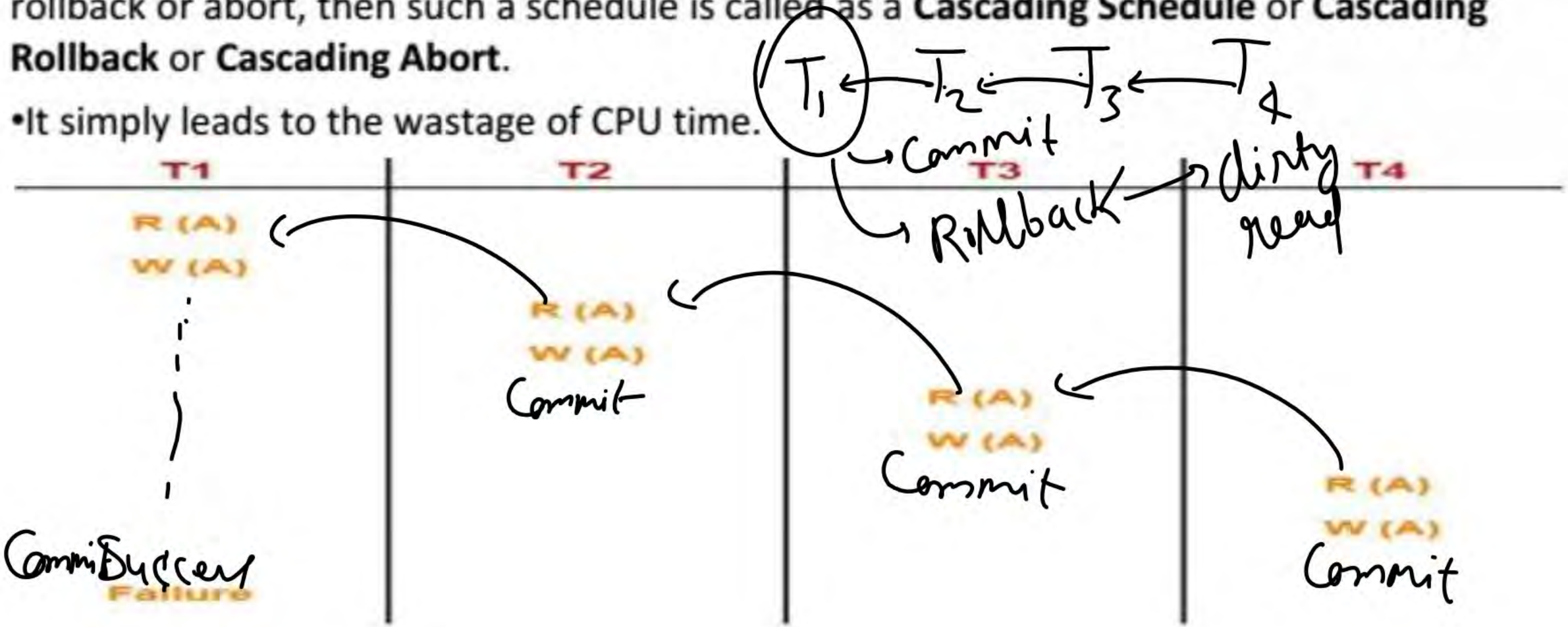
- Non-serial schedules which are not serializable are called as non-serializable schedules.
- Non-serializable schedules may be recoverable or irrecoverable.
- **Recoverable Schedules-**
- If in a schedule, A transaction performs a dirty read operation from an uncommitted transaction
- And its commit operation is delayed till the uncommitted transaction either commits or roll backs then such a schedule is called as a **Recoverable Schedule**



Topic:- Cascading Schedule

TEACHING WALLAH

- If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.
- It simply leads to the wastage of CPU time.



Cascading Recoverable Schedule

Topic:- Cascading Schedule

TEACHING WALLAH

Here,

- Transaction T2 depends on transaction T1.
- Transaction T3 depends on transaction T2.
- Transaction T4 depends on transaction T3

T1	T2	T3	T4
R (A)			
W (A)			
	R (A)		
	W (A)		
		R (A)	
		W (A)	
			R (A)
			W (A)
Failure			

Cascading Recoverable Schedule

Topic:- Cascading Schedule

TEACHING WALLAH

In this schedule,

- The failure of transaction T1 causes the transaction T2 to rollback.
- The rollback of transaction T2 causes the transaction T3 to rollback.
- The rollback of transaction T3 causes the transaction T4 to rollback.

T1	T2	T3	T4
R (A)			
W (A)			
	R (A)		
	W (A)		
		R (A)	
		W (A)	
			R (A)
			W (A)
Failure			

Cascading Recoverable Schedule

Topic:- Cascading Schedule

TEACHING WALLAH

Note:- If the transactions T2, T3 and T4 would have committed before the failure of transaction T1, then the schedule would have been irrecoverable.Such a rollback is called as a **Cascading Rollback**.

T1	T2	T3	T4
R (A)			
W (A)			
	R (A)		
	W (A)		
		R (A)	
		W (A)	
			R (A)
			W (A)
Failure			

Cascading Recoverable Schedule

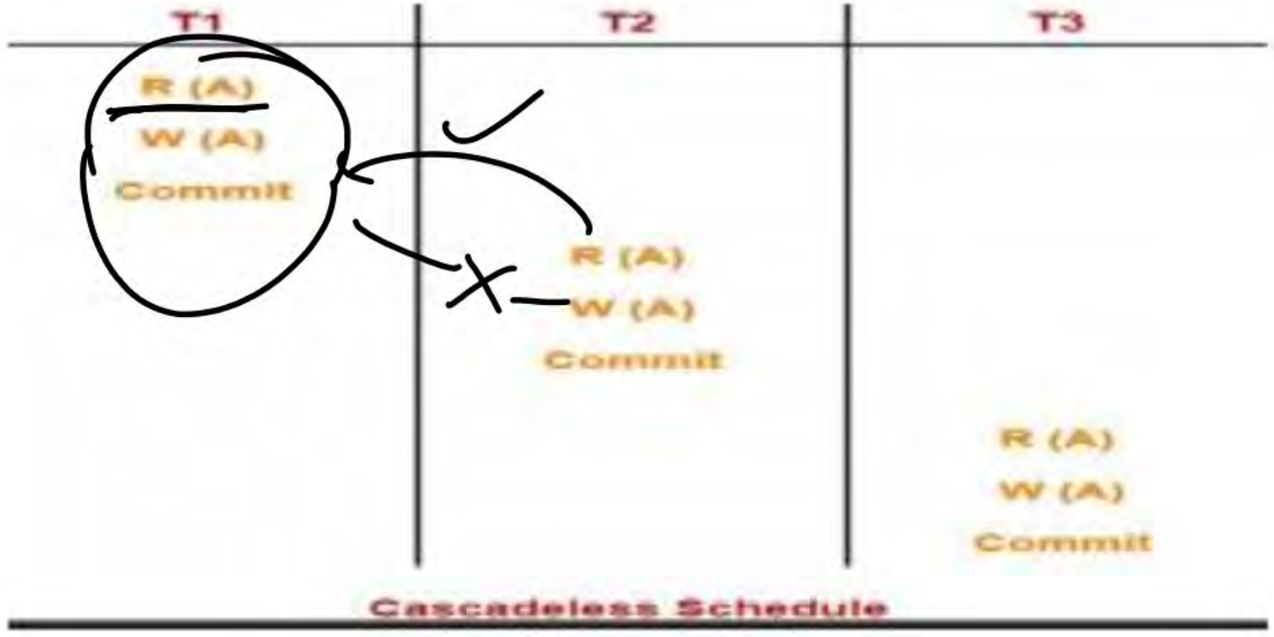
Topic:- Cascadeless Schedule-

TEACHING WALLAH

If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Cascadeless Schedule**.

In other words,

- Cascadeless schedule allows only committed read operations.
- Therefore, it avoids cascading roll back and thus saves CPU time.



Topic:- Cascadeless Schedule-

TEACHING WALLAH

If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Cascadeless Schedule**.

In other words,

- Cascadeless schedule allows only committed read operations.
- Therefore, it avoids cascading roll back and thus saves CPU time.

T1	T2	T3
R (A)		
W (A)		
Commit		
	R (A)	
	W (A)	
	Commit	
		R (A)
		W (A)
		Commit

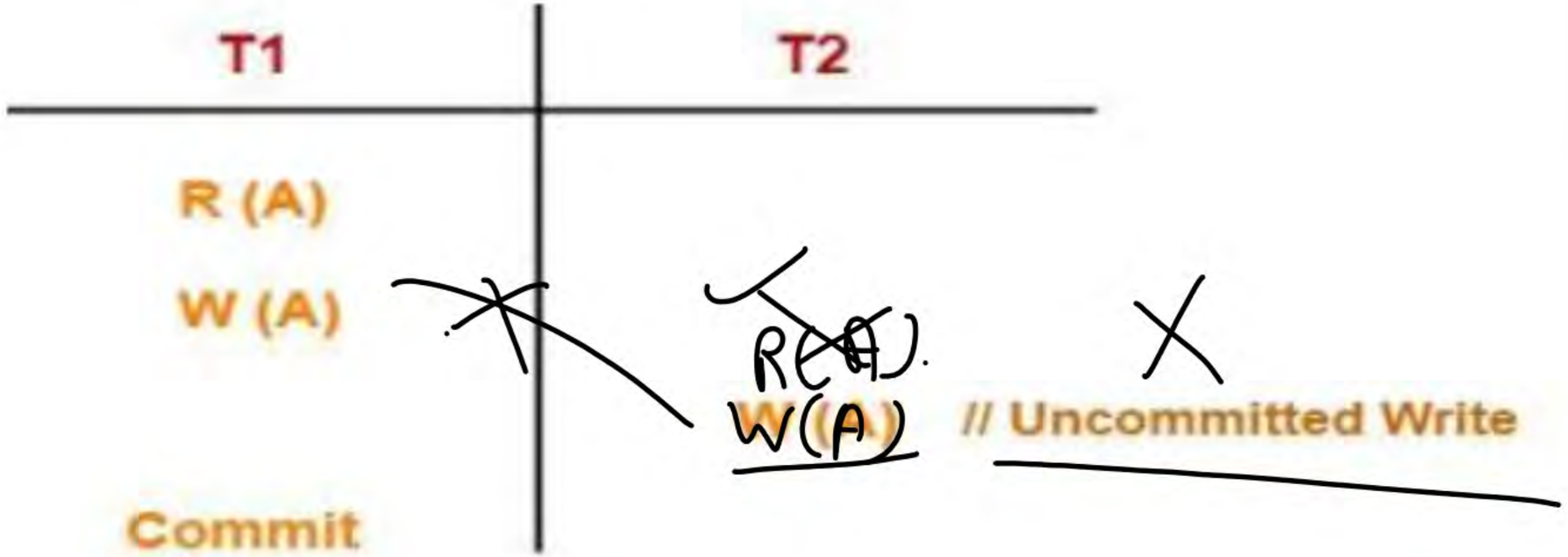
Cascadeless Schedule

Topic:- Cascadeless Schedule-

TEACHING WALLAH

NOTE-

- Cascadeless schedule allows only committed read operations.
- However, it allows uncommitted write operations.



Cascadeless Schedule

Topic:- Strict Schedule-

If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Strict Schedule**.

In other words,

- Strict schedule allows only committed read and write operations.
- Clearly, strict schedule implements more restrictions than cascadeless schedule.

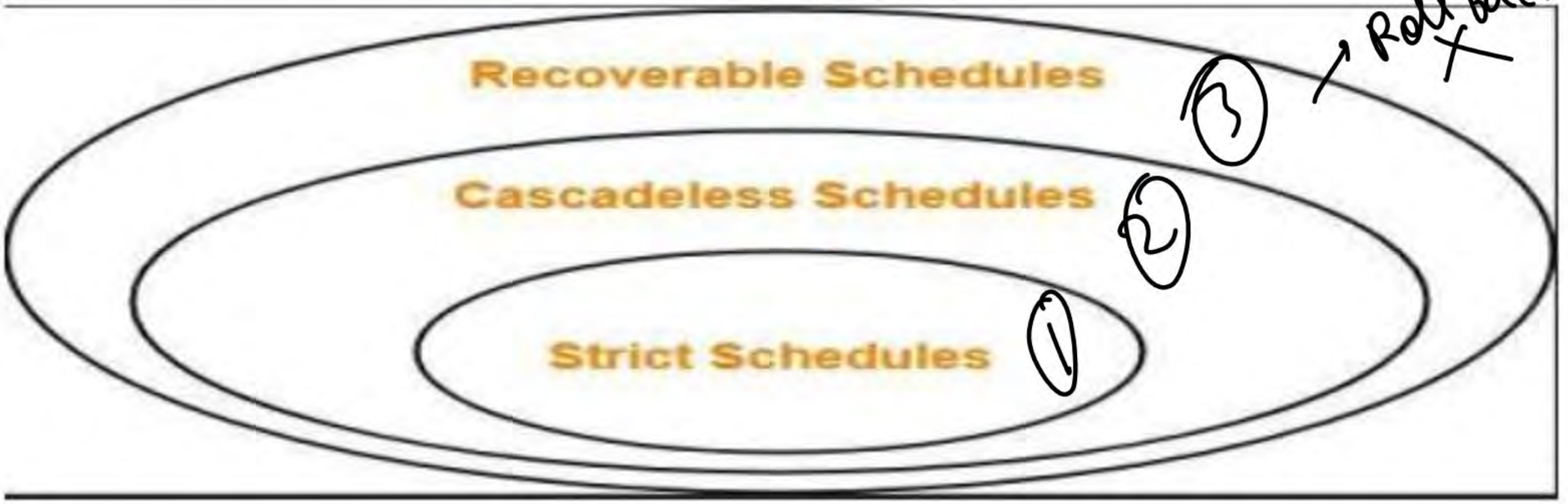
T1	T2
W (A)	
Commit / Rollback	
	R (A) / W (A)

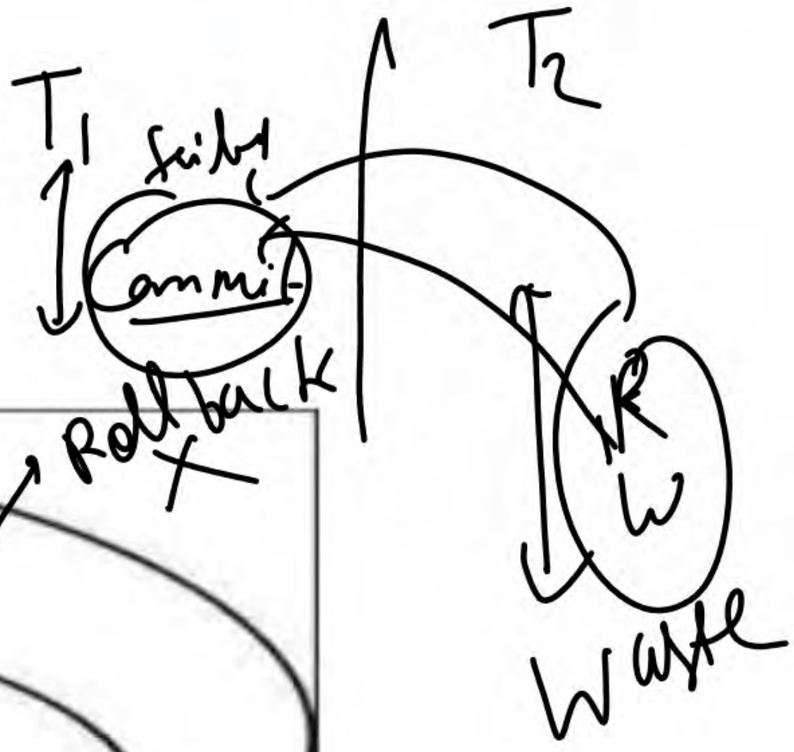
Strict Schedule

Topic:- Important Point on Recoverable Schdule

TEACHING WALLAH

- Strict schedules are more strict than cascadeless schedules.
- All strict schedules are cascadeless schedules.
- All cascadeless schedules are not strict schedules.

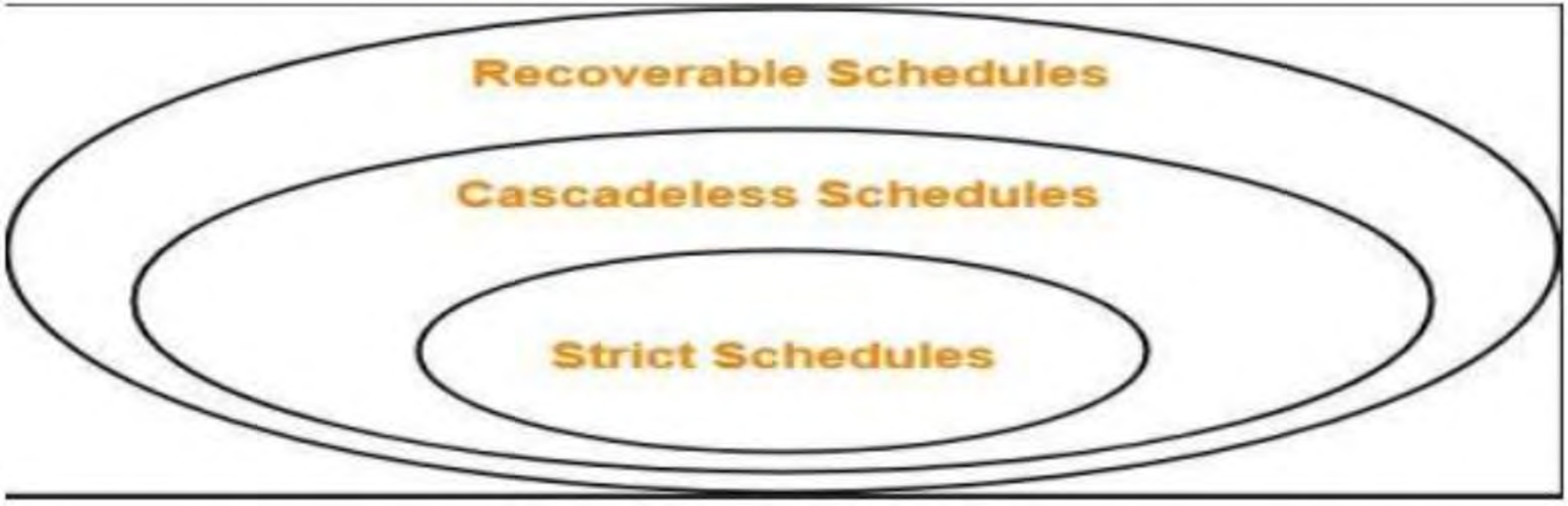




Topic:- Important Point on Recoverable Schdule

TEACHING WALLAH

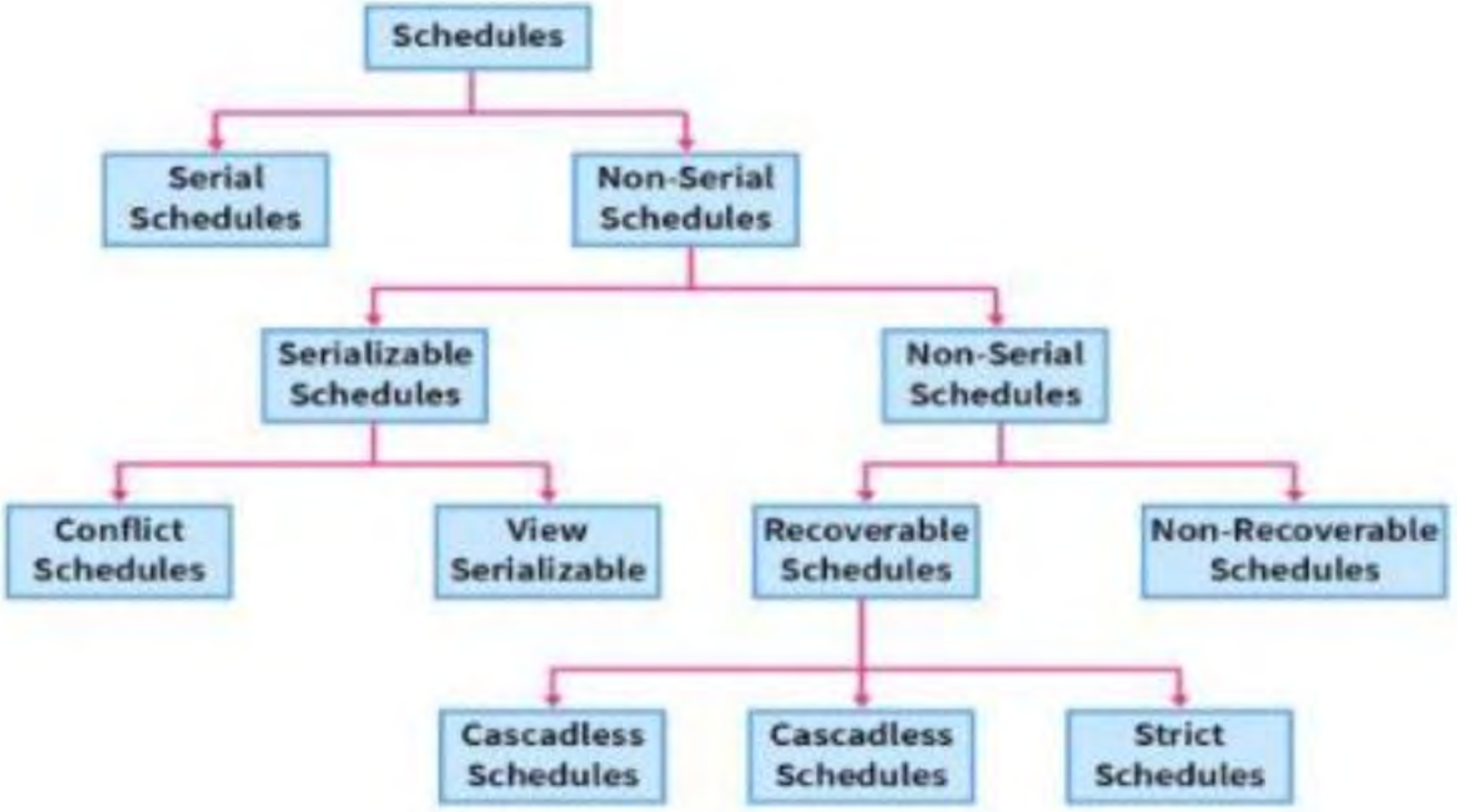
- Strict schedules are more strict than cascadeless schedules.
- All strict schedules are cascadeless schedules.
- All cascadeless schedules are not strict schedules.



Topic:- Short Revision on Types of Schedule

TEACHING WALLAH

Types of Schedules in DBMS



Topic:- Serializability

TEACHING WALLAH

What is a serializable schedule?

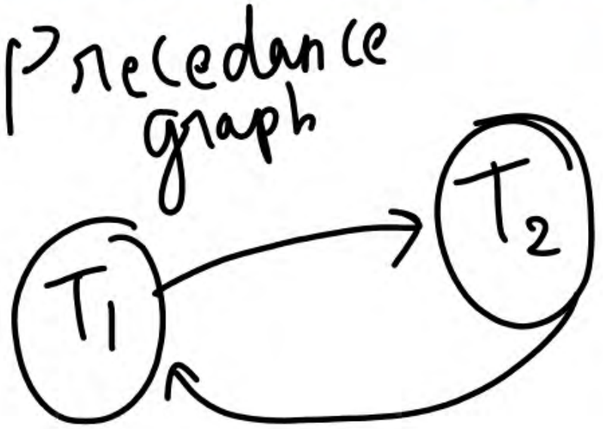
A non-serial schedule is called a serializable schedule if it can be converted to its equivalent serial schedule. In simple words, if a non-serial schedule and a serial schedule result in the same then the non-serial schedule is called a serializable schedule.

Testing of Serializability

To test the serializability of a schedule, we can use **Serialization Graph** or **Precedence Graph**. A serialization Graph is nothing but a Directed Graph of the entire transactions of a schedule.

It can be defined as a Graph G(V, E) consisting of a set of directed-edges E = {E1, E2, E3, ..., En} and a set of vertices V = {V1, V2, V3, ...,Vn}. The set of edges contains one of the two operations - READ, WRITE performed by a certain transaction.

Precedence Graph for Schedule S



Topic:- Serializability

NOTE:

If there is a cycle present in the serialized graph then the schedule is non-serializable because the cycle resembles that one transaction is dependent on the other transaction and vice versa. It also means that there are one or more conflicting pairs of operations in the transactions. On the other hand, no-cycle means that the non-serial schedule is serializable.

What is a conflicting pair in transactions?

To conclude, let's take two operations on data: "a". The conflicting pairs are:

1.READ(a) - WRITE(a)

2.WRITE(a) - WRITE(a)

3.WRITE(a) - READ(a)

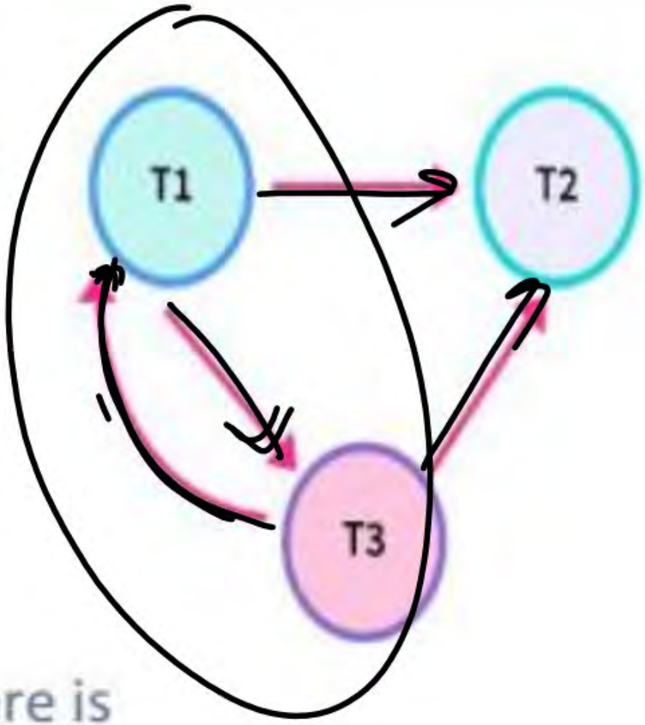
Note:

There can never be a read-read conflict as there is no change in the data.

Topic:- Example on Non-Serializability

TEACHING WALLAH

T1	T2	T3
R(x)		
		R(z)
		W(z)
	R(y)	
R(y)		
	W(y)	
		W(x)
	W(z)	
W(x)		



Non-serializable schedule. R(x) of T1 conflicts with W(x) of T3, so there is a directed edge from T1 to T3. R(y) of T1 conflicts with W(y) of T2, so there is a directed edge from T1 to T2. W(y\x) of T3 conflicts with W(x) of T1, so there is a directed edge from T3 to T. **Similarly, we will make edges for every conflicting pair. Now, as the cycle is formed, the transactions cannot be serializable**

Topic:- Conflict Serializable

TEACHING WALLAH

A non-serial schedule is a conflict serializable if, after performing some swapping on the non-conflicting operation results in a serial schedule**. It is checked using the non-serial schedule and an equivalent serial schedule. This process of checking is called Conflict Serializability in DBMS.**

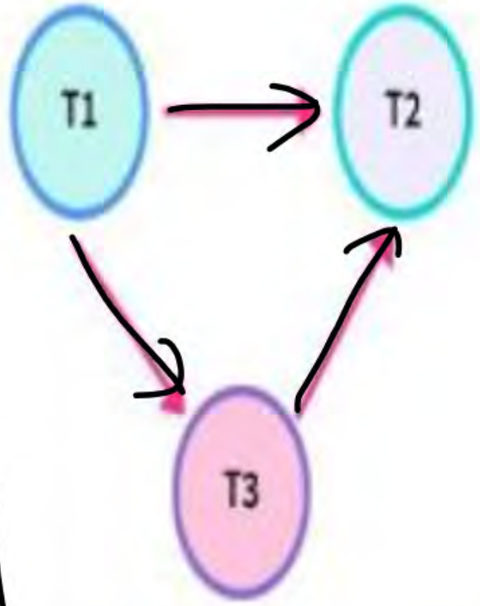
It is tedious to use if we have many operations and transactions as it requires a lot of swapping.

For checking, we will use the same Precedence Graph technique discussed above. First, we will check conflicting pairs operations(read-write, write-read, and write-write) and then form directed edges between those conflicting pair transactions. If we can find a loop in the graph, then the schedule is non-conflicting serializable otherwise it is surely a conflicting serializable schedule.

Topic:- Example on Conflict Serializable

TEACHING WALLAH

T1	T2	T3
R(x)		
	R(y)	
		R(y)
	W(y)	
W(x)		
		W(x)
	R(x)	
	W(x)	



As there is no loop in the graph(the graph is DAG), it is a conflict serializable schedule as well as a serial schedule. Since it is a serial schedule, we can detect the order of transactions as well.

The order of the Transactions: t1 will execute first as there is no incoming edge on T1. t3 will execute second as it depends on T1 only. t2 will execute at last as it depends on both T1 and T3.

So, order of its equivalent serial schedule is: t1 --> t3 --> t2

Topic:- Example on Conflict Serializable

TEACHING WALLAH

The order of the Transactions: t1 will execute first as there is no incoming edge on T1. t3 will execute second as it depends on T1 only. t2 will execute at last as it depends on both T1 and T3.

So, order of its equivalent serial schedule is: t1 --> t3 --> t2

T1 -> T2 ->T3

T1 -> T3 ->T2

T2-> T1->T3

T2->T3->T1

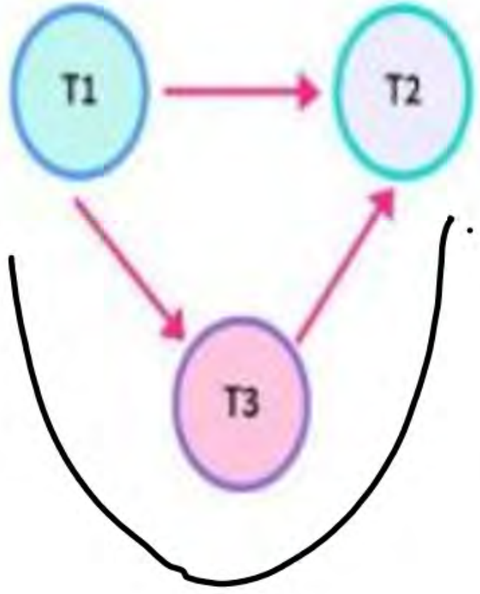
T3-> T1->T2

T3->T2->T1

!n = factorial of (n) = !3= 3x2x1=6

n!

(h = (3 = 3x2x1 = 6



NOTE:-

If a schedule is conflicting serializable, then it is surely a consistent schedule. On the other hand, a non-conflicting serializable schedule may or may not be serial. To further check its serial behavior, we use the concept of View Serializability.

Topic:- View Serializable

TEACHING WALLAH

View Serializability and View Serializable Schedules

If a non-serial schedule is **view equivalent** to some other serial schedule then the schedule is called View Serializable Schedule. It is needed to ensure the consistency of a schedule.

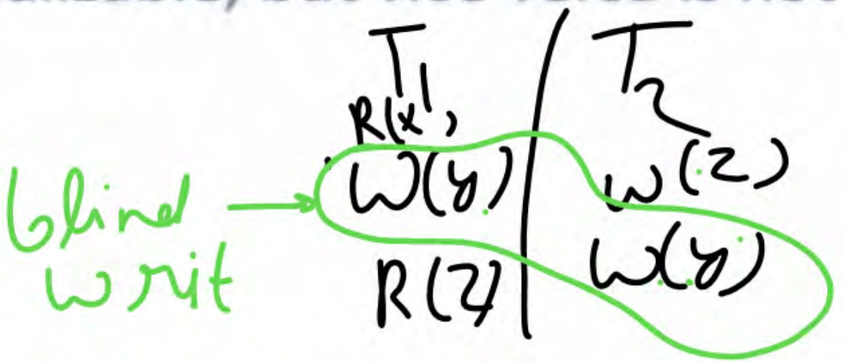
t1	t2
R(x)	
W(x)	
	R(x)
	W(x)
R(y)	
W(y)	
	R(y)
	W(y)

t1	t2
R(x)	
W(x)	
R(y)	
W(y)	
	R(x)
	W(x)
	R(y)
	W(y)

Topic:- View Serializable

NOTE:

A conflict serializable schedule is always viewed as serializable, but vice versa is not always true.



Actual process for checking view serializability

1.First, check for conflict serializability.

2.Check for a blind write. If there is a blind write, then the schedule can be view serializable. So, check its view serializability using the view equivalent schedule technique (stated above). If there is no blind write, then the schedule can never be view serializable.

Blind write is writing a value or piece of data without reading it.

Topic:- View Serializable

TEACHING WALLAH

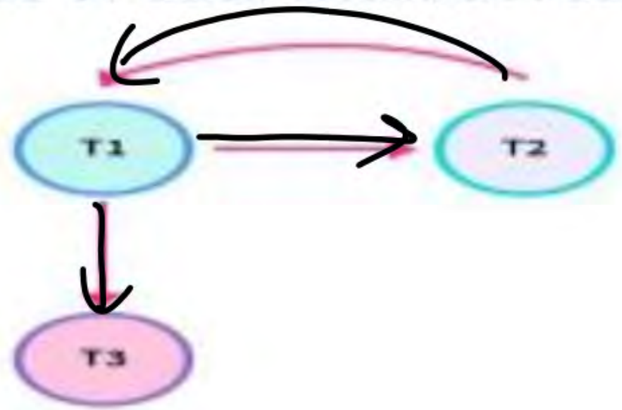
Actual process for checking view serializability

1.First, check for conflict serializability.

2.Check for a blind write. If there is a blind write, then the schedule can be view serializable. So, check its view serializability using the view equivalent schedule technique (stated above). If there is no blind write, then the schedule can never be view serializable.

Blind write is writing a value or piece of data without reading it.

t1	t2	t3
R(x)		
	W(x)	
W(x)		
		W(x)

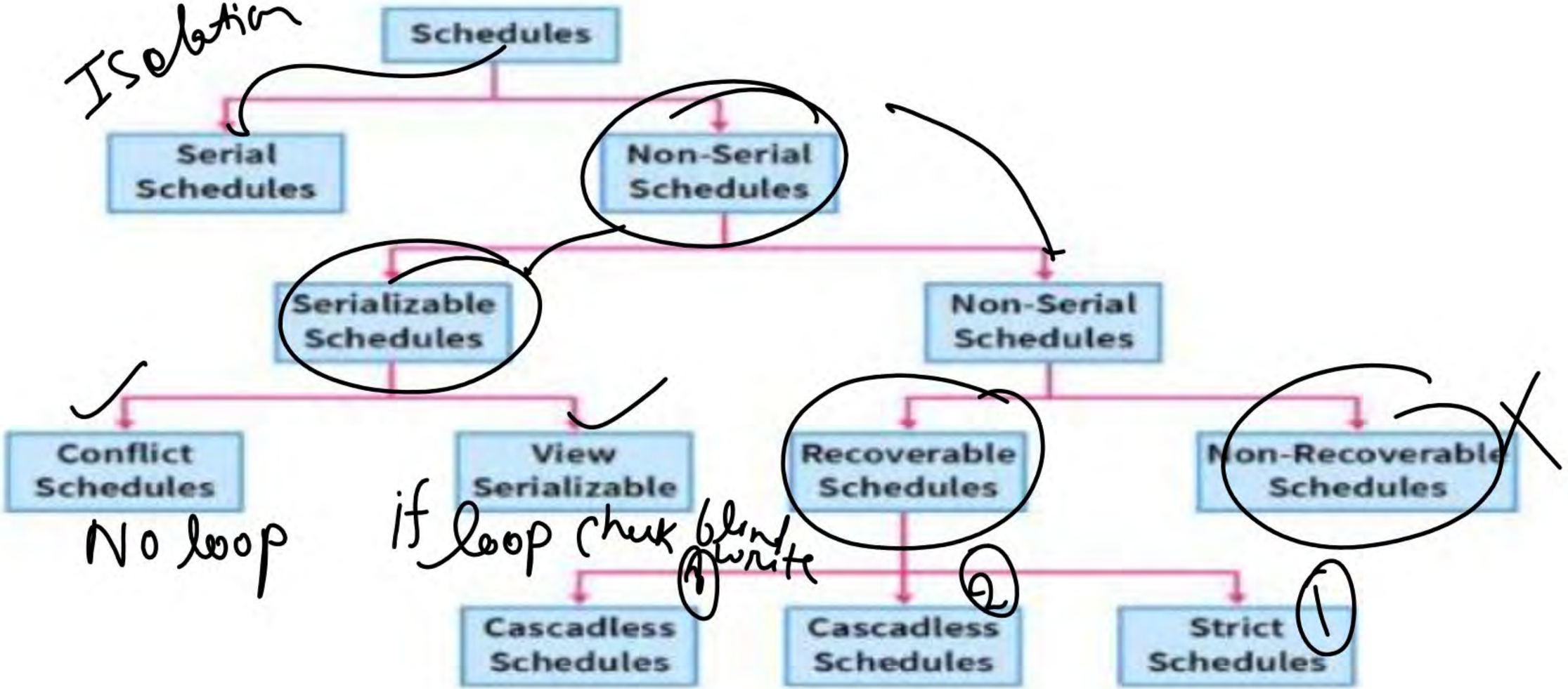


t1	t2	t3
R(x)		
W(x)		
	W(x)	
		W(x)

Topic:- Complete Revision on Schdule

TEACHING WALLAH

Types of Schedules in DBMS



TEACHING WALLAH

THANK YOU